

Article

Tamper-Evident Data and Model Provenance for IoT-Based Machine Learning Using Blockchain and Off-Chain Storage

Sangheetha Sukumaran ^{1,*} , Arun Korath ²  and Gowri Arun Menon ³

¹ College of Engineering & Technology, Fujairah University, Fujairah P.O. Box 2202, United Arab Emirates

² College of Business Administration, Kalba University, Kalba P.O. Box 11115, United Arab Emirates; arun.korath@ukb.ac.ae

³ Grade XII, St Mary's School, Fujairah P.O. Box 1665, United Arab Emirates; gowriarunn2008@gmail.com

* Correspondence: s.sukumaran@fu.ac.ae

Abstract

Machine learning models increasingly rely on continuously generated sensor data for automated decision-making in Internet of Things (IoT) environments. The distributed and often insecure nature of IoT infrastructures introduces risks related to data manipulation, lack of traceability, and unverifiable model evolution. Existing solutions typically address isolated aspects such as data security or access control but do not provide end-to-end provenance across the machine learning lifecycle. This paper proposes a tamper-evident data and model provenance framework for IoT-based machine learning that integrates blockchain with off-chain storage. The framework records cryptographic hashes and metadata of data, preprocessing outputs, and trained models on-chain while maintaining large artifacts off-chain to ensure scalability. Smart contracts establish verifiable linkage among lifecycle artifacts and automate provenance registration. The framework is evaluated in a simulated IoT-ML pipeline under integrity attack scenarios including data manipulation, model tampering, and metadata modification. Experimental results demonstrate reliable detection of unauthorized modifications with low verification latency and constant on-chain storage per record under controlled conditions. These findings indicate the feasibility of hybrid blockchain architectures for tamper-evident provenance in IoT-based machine learning systems, while highlighting the need for further validation in real-world deployments.

Keywords: blockchain; IoT security; tamper-evident provenance; data provenance; model provenance; smart contracts; off-chain storage; integrity verification



Academic Editors: Georgios Alexandridis, Thanasis Papaioannou, Georgios Siolas and Paraskevi Tzouveli

Received: 26 January 2026

Revised: 22 February 2026

Accepted: 25 February 2026

Published: 19 May 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

1.1. Background

The rapid growth of the Internet of Things (IoT) has led to large-scale deployment of networked sensors and smart devices that continuously generate data for intelligent decision-making. Machine learning (ML) models trained on IoT data are increasingly used in applications such as smart cities, healthcare monitoring, industrial automation, and intelligent transportation systems. However, the distributed, heterogeneous, and resource-constrained nature of IoT environments introduces significant trust and security challenges, even as ML enhances autonomy and operational efficiency [1].

IoT data is typically collected from diverse devices, transmitted through gateways, and processed in centralized or edge-based learning pipelines. This multi-stage lifecycle exposes IoT-based machine learning systems to threats such as data manipulation, unauthorized

injection, replay attacks, and selective modification during preprocessing or aggregation [2]. Even minor manipulation can degrade model performance or produce unsafe outcomes because machine learning models are highly sensitive to training data integrity. Conventional security mechanisms such as encryption and access control primarily protect data confidentiality but do not guarantee provenance or historical integrity once data enters the learning pipeline [3].

1.2. Challenges in IoT-Based Machine Learning Provenance

Beyond data integrity, the lack of transparency in machine learning workflows presents a major challenge. Trained models are often treated as opaque artifacts with limited traceability regarding data sources, preprocessing operations, and parameter configurations used during training. This lack of model provenance makes it difficult to verify whether models were retrained using authorized data or whether intermediate processes were modified [4]. The inability to audit model evolution undermines trust, accountability, and regulatory compliance, particularly as IoT-based ML systems increasingly support automated decision-making.

Blockchain technology has emerged as a potential solution to trust issues in distributed systems due to its decentralized architecture and immutability [5]. By maintaining an append-only ledger replicated across nodes, blockchain can provide tamper-resistant records without reliance on a central authority. Recent studies have explored blockchain for improving transparency in machine learning workflows [6] and enhancing IoT network security [7]. However, most existing approaches focus on isolated aspects such as secure data sharing or access control rather than lifecycle-level provenance.

Scalability is another critical challenge. Storing large IoT datasets or trained models directly on-chain is impractical due to storage limitations, transaction costs, and latency overheads [8]. Hybrid architectures combining off-chain storage with on-chain integrity verification have therefore been proposed. Nevertheless, many such systems fail to establish cryptographic linkage across data, preprocessing stages, and models, resulting in fragmented audit trails [9].

Another important gap is the limited attention to tamper-evidence as a design principle. While access control mechanisms aim to prevent unauthorized actions, tamper-evident systems ensure that any modification can be detected through cryptographic verification, enabling post-event auditing [10]. This capability is particularly important in IoT-based machine learning pipelines where artifacts traverse multiple administrative domains.

1.3. Research Objectives

This study aims to develop a unified framework that provides tamper-evident provenance across the entire lifecycle of IoT-based machine learning systems. The objectives are to:

- Establish cryptographic linkage between data, preprocessing outputs, and trained models;
- Enable detection of unauthorized modifications to lifecycle artifacts;
- Maintain scalability through hybrid on-chain/off-chain storage;
- Evaluate the feasibility of the approach under integrity attack scenarios.

1.4. Contributions

Inspired by these challenges, this paper proposes a tamper-evident data and model provenance framework for IoT-based machine learning using blockchain and off-chain storage. The framework stores large data and model artifacts off-chain for scalability while recording cryptographic hashes and metadata on-chain as an immutable provenance ledger.

By establishing verifiable linkage across the machine learning lifecycle, the framework enables integrity verification without excessive storage or computational overhead.

1.5. Organization of the Paper

The remainder of this paper is organized as follows. Section 2 reviews related work on blockchain-enabled IoT security, machine learning provenance, and hybrid architectures. Section 3 presents the system and threat models. Section 4 describes the proposed framework. Section 5 outlines the implementation and experimental setup. Section 6 discusses experimental results, followed by security and performance analysis in Section 7. Section 8 concludes the paper and outlines future research directions.

2. Related Work

The research related to this work spans the areas of blockchain-based IoT security, machine learning data and model provenance, blockchain integration with machine learning, and hybrid on-chain/off-chain architectures. In this section, the existing contributions are critically reviewed, and their limitations are highlighted with respect to tamper-evident provenance in IoT-based machine learning systems.

2.1. Blockchain for IoT Security and Trust

Blockchain has been intensively researched for use as a trust-enabling technology for IoT applications due to the decentralization and immutability characteristics of the former.

Initially, researchers proposed the use of blockchain for securing IoT communication, IoT device identity, and access control without the use of a central authority for IoT applications [11,12]. Dorri et al. presented a pioneer work for the integration of blockchain with IoT for improving the security and privacy of IoT applications through decentralization [6]. Christidis and Devetsikiotis also explored the use of smart contracts for automating IoT interactions [13].

Although these methods enhance the level of trust among entities in the IoT, they are mainly concerned with the issue of device-level security and sharing, and not the use of the IoT data for machine learning. Again, most methods ignore the issue of integrity attacks against the IoT datasets, without considering the impact of such attacks on the integrity of the ML model produced. This means they do not offer traceability on a lifecycle level for machine learning.

2.2. Data Provenance in Distributed Systems and IoT

Data provenance has been widely researched as a means to record information about the lineage and transformation history of digital objects. Hasan et al. presented secure models for data provenance to avoid forgery of transformation history in a distributed environment through cryptographic methods [9]. Then research on data provenance has been extended to clouds and scientific workflows [14].

For IoT, provenance techniques have been investigated to address source tracking and processing operations being performed on sensor data [15]. Nevertheless, most present-day IoT provenance systems either depend on centralized log servers or third-party services, which are both single points of failure, and address only data provenance and not machine learning model provenance.

2.3. Model Provenance and Machine Learning Lifecycle Management

Provenance in machine learning refers to the process of recording how a machine learning system was trained on certain datasets and hyperparameters. Sculley et al. have cited the dangers of changes in machine learning going unrecorded and termed them as a

kind of ‘technical debt’ [4]. Recent studies have stressed the need for reproducibility and ‘traceability’ in machine learning systems, especially in a controlled environment [16].

Some systems try to follow the tracking of ML experiments and model versions using centralized repositories and metadata systems. Though useful for increased reproducibility, these systems rely on a trusted execution environment and lack the ability to provide cryptographic assurance of integrity. They are therefore inadequate for adversarial and multi-stakeholder IoT domains where trust assumptions are lowest.

2.4. Blockchain-Enabled Machine Learning

The integration of blockchain technology with machine learning has also garnered much attention lately, as it provides a way of improving the transparency, trust, and decentralization of machine learning. Various research works have proposed using blockchain-based federated learning frameworks to securely manage the coordination of model aggregations in a distributed manner for the development of trustworthy models [17,18]. Blockchain-based marketplaces have also been discussed for data/models, where the ownership can also be traced [19].

Despite these breakthroughs, the majority of existing work on the use of blockchain for ML is dedicated to training orchestration or aligning incentives, as opposed to comprehensive data provenance tracking. Data integrity, as well as model integrity, is implicitly assumed rather than explicitly checked. Finally, a lack of connection between the data, the processing, and the models is a problem for these frameworks, as they are ill-suited to handle auditable processes.

2.5. Hybrid On-Chain and Off-Chain Architectures

To address scalability limitations of blockchain systems, hybrid architectures have been proposed that combine on-chain integrity verification with off-chain storage of large data objects. In these approaches, blockchain stores cryptographic hashes and metadata, while actual datasets and models are maintained in external repositories such as cloud storage, distributed file systems, or content-addressable networks such as IPFS [8,11]. Several studies have demonstrated the effectiveness of hybrid designs in reducing storage overhead while preserving tamper-evidence [6,8]. For example, IPFS-based systems enable decentralized storage with content-addressable retrieval, allowing verification of file integrity through hash comparison [11]. Similar hybrid frameworks have been applied to secure data sharing, digital asset management, and machine learning model distribution [12,13].

Despite these advances, many existing solutions treat data and models as independent entities and do not maintain explicit linkage across preprocessing steps and training configurations. Consequently, they provide only partial provenance and limited auditability. The proposed framework addresses this gap by establishing cryptographic dependencies across all stages of the machine learning lifecycle, ensuring end-to-end traceability while maintaining scalability.

2.6. Machine Learning in IoT Systems

Machine learning techniques are increasingly deployed in IoT systems to enable predictive analytics, anomaly detection, and automated control [1,20]. Due to the resource constraints of IoT devices, machine learning pipelines often involve multiple stages, including data collection at edge nodes, preprocessing, centralized or distributed model training, and deployment for inference. The distributed nature of IoT introduces challenges such as unreliable connectivity, heterogeneous data sources, and security vulnerabilities [11].

Data integrity becomes particularly critical because compromised training data can significantly degrade model performance. Furthermore, the absence of transparent provenance mechanisms makes it difficult to audit model evolution or verify compliance with

operational requirements [14]. Recent research has explored approaches such as edge-based learning, federated learning, and secure data sharing in IoT environments [17,18]. However, provenance tracking across the entire lifecycle remains an open problem. This motivates the need for frameworks that can provide verifiable traceability of data transformations and model updates in IoT-based machine learning systems.

2.7. Comparison with Existing Approaches

Several prior studies have explored blockchain-based security and provenance mechanisms for IoT and machine learning systems [6,8,15,17–19] (Table 1). However, these approaches typically focus on isolated aspects such as secure data sharing, federated learning coordination, or model marketplaces rather than lifecycle-level provenance.

Table 1. Comparison with representative prior works [6,8,15,17–19].

Feature	IoT Provenance Systems [15]	Blockchain-ML Frameworks [17,18]	Hybrid Storage Systems [8]	Proposed Framework
Lifecycle coverage	Data only	Training coordination	Partial	Full
Data–model linkage	No	Limited	No	Yes
Tamper evidence	Limited	Moderate	Moderate	Strong
Scalability	Moderate	Moderate	High	High
IoT suitability	Moderate	Limited	Moderate	Targeted

2.8. Summary and Research Gap

In conclusion, there is existing work that shows the viability of the use of blockchain for improving trust in the context of IoT and machine learning. Nevertheless, the existing works have one or more of the following drawbacks: (i) decoupling data and model provenance, (ii) non-tamper-evident security, (iii) use of trust assumptions for the central entities, and (iv) unscalability for IoT-sized data. The paper closes this gap with the design of a tamper-evident unified provenance for the IoT data, preprocessing, and machine learning models within a hybrid blockchain architecture.

3. System Model and Threat Model

This section introduces the system architecture, entities involved in the system, trust assumptions, and capabilities of potential adversaries that this study takes into account. It is critical to define the system and threat models succinctly in order to discuss and analyze the efficacy of proposed tamper-evident provenance concepts and systems for their applicability and security.

The system architecture diagram is shown in Figure 1.

3.1. System Model

The considered system represents a typical IoT-based machine learning pipeline augmented with blockchain-based provenance tracking and off-chain storage. The system consists of the following entities:

IoT Devices:

A stream of data is continuously generated from various distributed IoT sensors and smart devices. Every device is uniquely identifiable and generates time-stamped measurements. To be consistent with typical deployments of IoT, it is assumed that the devices have limited capabilities in both computation and storage.

Edge or Gateway Nodes:

The data from the IoT sensors is received by the edge or gateway nodes, which execute initial processes related to filtering, normalization, aggregation, or feature extraction. These devices act as middle nodes in the conduction of machine learning processes from the IoT sensors.

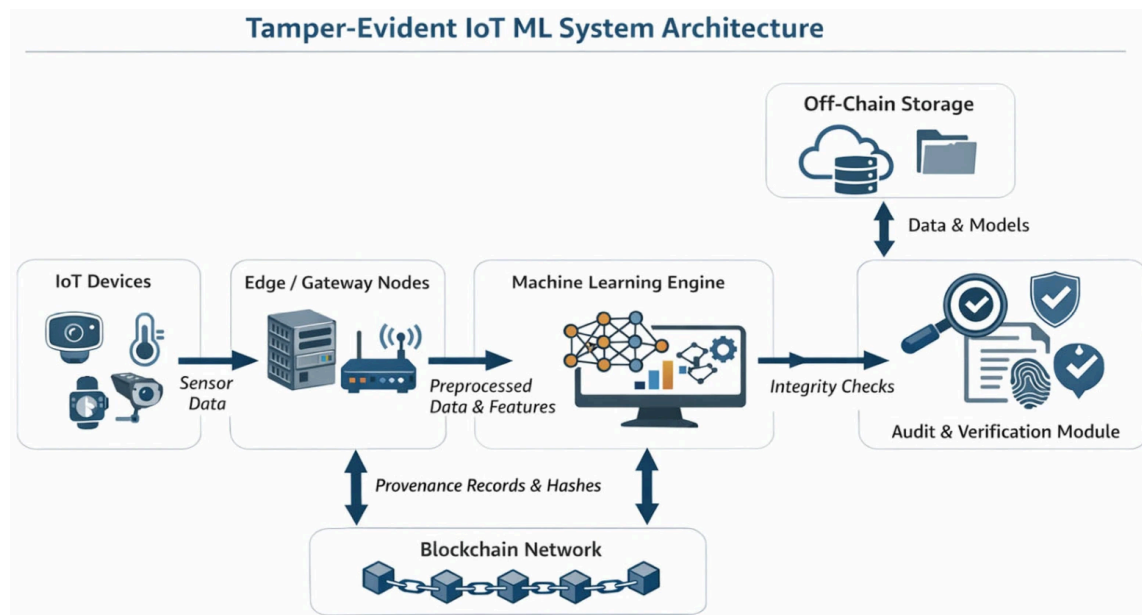


Figure 1. Tamper-evident IoT–ML system architecture showing data flow from IoT devices through edge processing to the machine learning engine, with blockchain-based provenance for integrity and off-chain storage for data and models. The audit and verification module performs integrity checks using on-chain records. Arrows indicate data and provenance flows between components.

Off-Chain Storage:

Because of the quantity and speed of the IoT, the raw sensor data, processed data, and machine learning models are all stored in the off-chain. However, the off-chain storage can include cloud storage, distributed file storage, and content addressable storages such as IPFS. The off-chain is expected to be available but not trusted.

Machine Learning Engine:

The ML Engine is responsible for model training, validation, retraining, and deployment based on data extracted from off-chain data storage. It can function in a centralized or distributed manner and “must check the integrity of the data/model before use.

Blockchain Network:

The permissioned/consortium blockchain is associated with an immutable ledger for storing the hashes and metadata of data batches, preprocessing tasks, and the trained models themselves. Smart contracts provide the rules for registering provenance and keep track of versioned records too. The blockchain nodes are trusted to follow the rules of the consensus protocol for achieving immutability and fault tolerance [5].

Audit and Verification Module:

This component verifies data and model integrity by recomputing cryptographic hashes from off-chain artifacts and comparing them with on-chain records. It enables detection of tampering and supports forensic auditing.

3.2. Provenance Scope

The system tracks provenance at three levels to ensure comprehensive traceability across the machine learning lifecycle. First, data provenance captures the origin, identity, and integrity of IoT data batches. Second, process provenance records preprocessing operations and feature extraction parameters applied during data preparation. Third, model

provenance documents training datasets, model versions, and configuration metadata associated with model development. All provenance records are cryptographically linked to enable end-to-end traceability across the lifecycle.

3.3. Trust Assumptions

The framework operates under several trust assumptions consistent with prior blockchain-based security studies [6,8]. It is assumed that blockchain consensus mechanisms and underlying cryptographic primitives remain secure and that smart contracts are correctly implemented and deployed. IoT devices are considered potentially vulnerable to compromise; however, adversaries are assumed unable to break standard cryptographic hash functions. Off-chain storage is treated as untrusted and may behave maliciously or exhibit faults. Additionally, the machine learning engine is assumed to follow prescribed verification protocols before initiating training or inference processes.

3.4. Threat Model

The adversary is assumed to be computationally bounded but capable of launching realistic attacks targeting IoT-based machine learning systems. The threat scenarios considered in this study are illustrated in Figure 2.

Threat Model for IoT-Based Machine Learning Systems

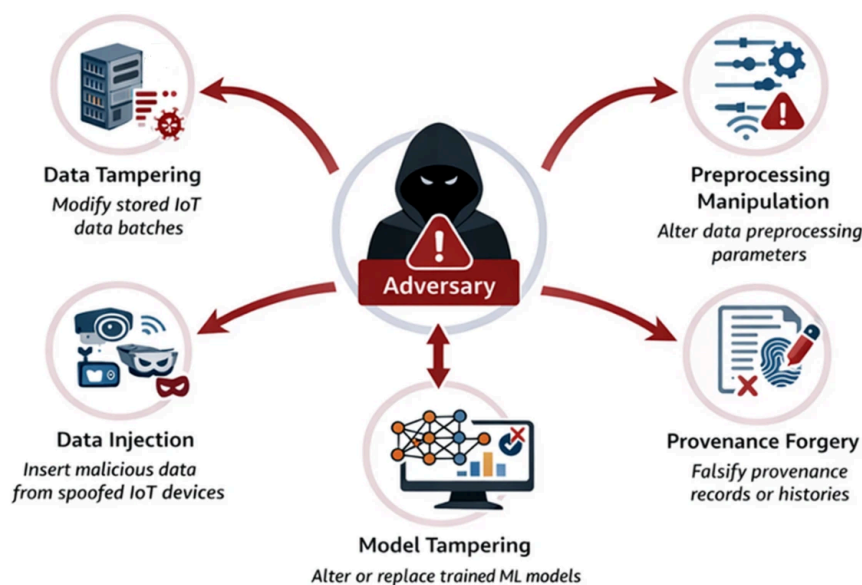


Figure 2. Threat model illustrating potential attack vectors in IoT-based machine learning systems. Arrows indicate possible attack paths from the adversary to system components.

Data tampering attacks involve unauthorized modification of IoT data stored off-chain through value alteration, noise injection, or selective deletion of records, with the objective of corrupting training data and degrading model performance [2]. In data injection attacks, malicious data originating from unauthorized or spoofed IoT devices is introduced into the pipeline to influence model training, including fabricated sensor readings that mimic legitimate formats.

Preprocessing manipulation occurs when an adversary alters preprocessing parameters or feature extraction configurations at edge nodes, thereby modifying the input distribution used for training. Model tampering attacks target trained models after the training phase by altering stored parameters or replacing the model with a malicious

version, compromising inference integrity and deployment trustworthiness. Provenance forgery attempts involve manipulation or deletion of historical records to conceal prior tampering activities, such as rewriting logs or replacing earlier versions of data or models.

3.5. Threats Outside Scope

Certain threats are considered outside the scope of this work to enable focused evaluation of provenance and tamper-evidence mechanisms, consistent with related studies [17]. These include denial-of-service attacks against blockchain or storage infrastructure, side-channel attacks targeting IoT hardware, cryptographic key compromise at blockchain nodes, and adversarial machine learning attacks such as evasion during inference.

3.6. Security Goals

Based on the system and threat models, the proposed framework aims to achieve several security objectives. First, tamper-evidence ensures that any unauthorized modification to data, preprocessing steps, or models can be detected. Second, traceability guarantees that all artifacts remain verifiable across the machine learning lifecycle. Third, integrity verification enables validation of data and models before use. Finally, scalability ensures that the framework can support large-scale IoT data without requiring excessive on-chain storage. These goals guide the design of the framework presented in the subsequent section.

4. Proposed Tamper-Evident Provenance Framework

This section presents the proposed framework for achieving tamper-evident data and model provenance in IoT-based machine learning systems. The framework is designed to ensure end-to-end traceability, integrity verification, and auditability across the entire machine learning lifecycle while maintaining scalability through a hybrid on-chain/off-chain architecture.

4.1. Design Principles

The proposed framework is guided by the following design principles:

1. **Tamper-Evidence over Prevention:**
Instead of trying to secure against all unauthorized activities, the system provides the assurance that any change to the information or processes will be cryptographically recognizable.
2. **Lifecycle-Level Provenance:**
The provenance information needs to be accounted for throughout all levels of the machine learning process, from generation to preprocessing, from extraction to model training, and from model updates.
3. **Separation of Storage and Verification:**
Large data objects are kept off-chain, and integrity proofs and metadata are maintained on-chain to ensure scalability.
4. **Decentralized Trust:**
Trust on the blockchain network is spread among the nodes, hence the absence of the need to log or prove with a central authority.

4.2. Framework Overview

The architecture combines the use of a blockchain for provenance storage and an off-chain data repository with cryptographic links between the two. Every artifact within the ML process, from data batches through preprocessing results to trained models, is distinguished by a cryptographic hash.

An artifact refers to a data batch, a preprocessing result, or a trained model within the ML process. Any change to these artifacts will cause the hash value to change, and consequently, the digital fingerprint stored within the blockchain will also change

The framework operates in four stages:

1. IoT data registration;
2. Preprocessing and feature provenance;
3. Model provenance registration;
4. Verification and auditing.

Each stage is described in detail below.

4.3. IoT Data Provenance Registration

Before detailing the registration workflow, we define the provenance artifacts, hash inputs, and on-chain metadata schema used throughout the framework as listed in Tables 2 and 3.

Table 2. Hash Inputs.

Artifact	Symbol	Hash Input (Exact)	Hash Function
Raw IoT batch file	D_i	bytes of canonical stored batch file	SHA-256
Feature output file	$F_{i,k}$	bytes of canonical stored feature file	SHA-256
Preprocessing spec	$\text{Spec}(P_k)$	canonical JSON of steps + params + code ref	SHA-256
Training configuration	TrainCf_{g_j}	canonical JSON of model params + seeds + split	SHA-256
Serialized model file	M_j	bytes of saved model file	SHA-256
Evaluation report	Eval_j	canonical JSON of metrics + test id	SHA-256

Table 3. On-chain Metadata.

Record Type	Required Fields Stored On-Chain
Data record	data_id, device_id, time window, hash_data, uri_data
Process record	process_id, input_hash, output_hash, process_spec_hash, uri_features, timestamp
Model record	model_id, version, model_hash, training_cfg_hash, feature_hashes, eval_hash, uri_model, timestamp

Canonical storage refers to deterministic serialization with fixed field ordering, encoding, and formatting to ensure uniform hash computation across environments.

IoT data generated by devices are grouped into logical batches based on time windows or device identifiers. For any given data batch D_i , a cryptographic hash is computed:

- $H(D_i) = \text{Hash}(D_i)$
where $\text{Hash}(\cdot)$ denotes a collision-resistant hash function such as SHA-256.

The batch D_i is stored off-chain, and the blockchain has the following metadata:

- Data batch identifier;
- Hash $H(D_i)$;
- Device identifier(s);
- Timestamp;
- Off-chain storage pointer.

These fields correspond to the data record schema defined in Table 3.

This registration generates an unchangeable link between the batch of data and their point of origin. The verification stage will be affected if changes occur to any of this data held in storage.

4.4. Process and Feature Provenance

Raw IoT data undergoes some preprocessing and feature extraction, which are necessary transformations for model behavior, hence should also be tracked as part of provenance.

For each preprocessing operation P_k applied to data batch D_i , the resulting feature set $F_{i,k}$ is generated. The framework computes:

$$H(F_{i,k}) = \text{Hash}(F_{i,k})$$

A hash of preprocessing parameters and code configuration is also recorded to capture transformation semantics. The blockchain stores:

- Input data hash $H(D_i)$;
- Output feature hash $H(F_{i,k})$;
- Processing parameters hash;
- Operation identifier.

By linking feature hashes to their originating data hashes, the framework establishes a verifiable transformation chain. Any unauthorized changes to preprocessing steps or the parameters become detectable.

4.5. Model Provenance Registration

Machine learning models are trained using verified feature sets retrieved from off-chain storage. For each trained model M_j , the framework computes:

$$H(M_j) = \text{Hash}(M_j)$$

The blockchain records the model provenance metadata, including:

- Model identifier and version;
- Model hash $H(M_j)$;
- Hashes of training feature sets;
- Training configuration and hyperparameter hash;
- Evaluation metrics hash.

This registration cryptographically binds the trained model to the exact data and configurations used during training. As a result, any attempt to retrain, replace, or change a model without updating provenance records becomes immediately detectable.

4.6. Smart Contract Enforcement

Smart contracts govern provenance registration and ensure the consistency of records across the system. The contracts enforce mandatory registration of data before use, validation of input hashes during model registration, and versioning through immutable record updates. They also maintain immutable linkage among data, processing steps, and models throughout the machine learning lifecycle. Rather than storing large data objects on-chain, the smart contracts maintain lightweight metadata structures, thereby minimizing blockchain storage overhead [8].

4.7. Verification and Auditability

Before data or models are used for training or inference, the verification module recomputes cryptographic hashes from off-chain artifacts and compares them with corresponding on-chain records. Any mismatch indicates potential tampering or unauthorized modification.

The immutable blockchain ledger further enables retrospective auditing of machine learning workflows, identification of compromised data or models, and reconstruction

of the complete lifecycle history of artifacts. This capability supports forensic analysis, regulatory compliance, and trust assurance in multi-stakeholder IoT environments.

5. Implementation and Experimental Setup

This section focuses on the implementation and environment setup of the proposed tamper-evident system framework in evaluating the proposed provenance system framework. The experiment aims at analyzing the feasibility of the framework and its efficacy in detecting violation of integrity, besides analyzing the overhead cost.

5.1. Experimental Environment

The implementation of the proposed framework employs a combination of widely used open-source tools commonly adopted in blockchain and machine learning research. All experiments were conducted within a virtualized environment designed to simulate a real-world IoT-based machine learning scenario without requiring physical sensor deployment.

The experimental environment consists of a local machine learning pipeline implemented in Python 3.12, a local blockchain test network for provenance recording, an off-chain storage system for data and model artifacts, and a verification module for integrity and provenance checks. This configuration enables repeatable experimentation while isolating the impact of the provenance framework from variability introduced by external infrastructure.

5.2. Dataset and IoT Data Simulation

A synthetic dataset has been created programmatically to represent the data that might be IoT-generated by imitating multi-dimensional sensor readings that are generally seen in IoT scenarios. Each dataset has numeric telemetry fields and labels created for the purpose of supervised learning. Although synthetic, the distribution and structure of the data were inspired by publicly available IoT datasets such as TON_IoT [21], making it appropriate for provenance tracking and integrity evaluation. Since the evaluation focuses on integrity, provenance linkage, and tamper detection rather than on task-specific prediction accuracy, synthetic data suffices to preserve realistic attack surfaces since integrity attacks target data manipulation, injection, and replacement independently of semantic data content.

To simulate multiple IoT devices, the dataset is partitioned by assigning device identifiers and gathering the records in time-based batches. Each batch corresponds to a single logical IoT data unit D_i generated within a fixed time window. This batching strategy reflects common practices regarding data ingestion in IoT systems and supports scalable provenance registration.

Synthetic datasets were used to enable controlled injection of integrity attacks while isolating provenance verification mechanisms from domain-specific variability. Since tampering detection relies on structural modification of artifacts rather than semantic content, synthetic data provides a suitable test environment. Validation using real IoT datasets is identified as future work to assess deployment readiness.

5.3. Machine Learning Models

The machine learning part is achieved through conventional models of supervised machine learning. These models of machine learning are widely utilized in analytics related to IoT. The models of machine learning utilized are determined based on the interpretability of models and computational simplicity. These models do not depend on optimization.

For each experiment:

- Input features are derived through deterministic pre-processing steps.
- Models are trained using verified feature sets only.

- Trained models are serialized and stored as off-chain artifacts.

Model evaluation metrics such as accuracy and F1-score are computed to ensure that the provenance mechanisms do not adversely affect baseline learning performance.

5.4. Off-Chain Storage Configuration

All raw IoT data batches, processed sets of features, as well as machine learning models, are kept off-chain. Off-chain storage is achieved using a file system repository on a local disk, which simulates the operation of a distributed environment. Every off-chain artifact is assigned a distinct identity, along with a storage pointer.

Prior to storage, each artifact undergoes the computation of a cryptographic hash via a collision-resistant hash function. The cryptographic hash is then used as a integrity fingerprint and is stored in the blockchain, along with metadata. The off-chain storage layer is modeled as untrusted, as is typically the case [8].

5.5. Blockchain and Smart Contract Setup

We used a local Ethereum dev chain (Hardhat) to emulate a permissioned setting. Smart contracts are implemented to manage provenance registration and metadata storage. The contracts support the following operations:

- Registration of IoT data batch hashes and metadata.
- Registration of preprocessing and feature hashes.
- Registration of trained model hashes and training configurations.
- Retrieval of provenance records for verification.

The blockchain stores only lightweight metadata and cryptographic hashes, ensuring minimal storage overhead. No raw data or model parameters are stored on-chain.

5.6. Attack Simulation

Controlled integrity attacks, which are designed to reflect realistic adversarial actions without assuming advanced cryptographic capabilities, are injected at various points in the pipeline in order to assess tamper-evidence.

The simulated attacks include:

- Data Tampering: IoT data batches stored are modified after provenance registration.
- Data injection: Introduction of unauthorized data batches into the pipeline.
- Preprocessing manipulation: Alteration of feature extraction parameters.
- Model tampering: Modification of serialized trained models stored off-chain.

Each attack scenario is applied independently to isolate its impact on provenance verification.

5.7. Verification Procedure

For each experimental run, the verification module retrieves off-chain artifacts and recomputes their cryptographic hashes. These hashes are compared with corresponding on-chain records. Any mismatch is flagged as a provenance violation.

The verification process is executed:

- Before model training;
- Before model deployment;
- After simulated attacks.

This ensures continuous integrity validation across the ML lifecycle.

5.8. Evaluation Metrics

The framework is evaluated using the following metrics:

- Tamper detection rate, defined as the proportion of successful integrity violations detected.
- Verification latency, measuring the time required to perform hash computation and on-chain comparison.
- Blockchain storage overhead, quantified as metadata size per provenance record.
- Machine learning performance impact, measured by comparing baseline and verified training results.

These metrics collectively assess security effectiveness and system overhead.

6. Results and Discussion

The experimental results of the proposed tamper-evident provenance framework and the discussions of these results with regard to integrity of provenance, capability of detecting tamper attempts, overhead caused by the proposed approach, and effect of proposed approach on machine learning performance are described in this section.

6.1. Provenance Integrity and Tamper Detection

The main goal of the proposed framework is to ensure integrity verification of the tamper-evident nature throughout the machine learning process in the IoT environment. Experiments reveal the success of the framework in identifying all integrity violations introduced in the system.

For IoT data batches that were stored off-chain and were then altered after the registration of their provenance on the blockchain, the verification module reliably detected mismatches between the computed hashes and those recorded on the blockchain. This serves to confirm that even slight modifications to the data had been possible via the link established through the blockchain between the off-chain and on-chain components. Though the current assessment is on the data, model, and metadata artifacts, the framework architecture lends itself to incorporating the hash validation mechanism on the intermediate outputs of the preprocessing phase as well.

Model tampering attacks, such as manipulation of serialized models, were also accurately detected. Since models were cryptographically tied to training data and metadata, any illegal manipulation would render the stored hash of the model invalid. The above findings demonstrate that the proposed framework successfully supports tamper evidence for multiple artifact types—training data, model, and metadata, as opposed to single integrity points.

Table 4 below shows the results of the integrity verification, where the detection of tamper with exact precision, without a single false positive in our experiments, is achieved. Gas refers to the provenance registration transaction.

Table 4. Provenance verification results under different attack scenarios.

Scenario	Tamper Detected Component	Total Verification Time (ms)	Gas Used
NoTamper	None	31.33	142,400
DataTamper	Data	26.52	142,400
ModelTamper	Model	23.66	142,400
MetaTamper	Metadata	22.96	142,400

Figure 3 depicts the tamper detection results under various scenarios of attacks. The results of the experiment indicate that the approach was able to correctly identify all the injected integrity violations on the various artifacts related to the data, model, and metadata.

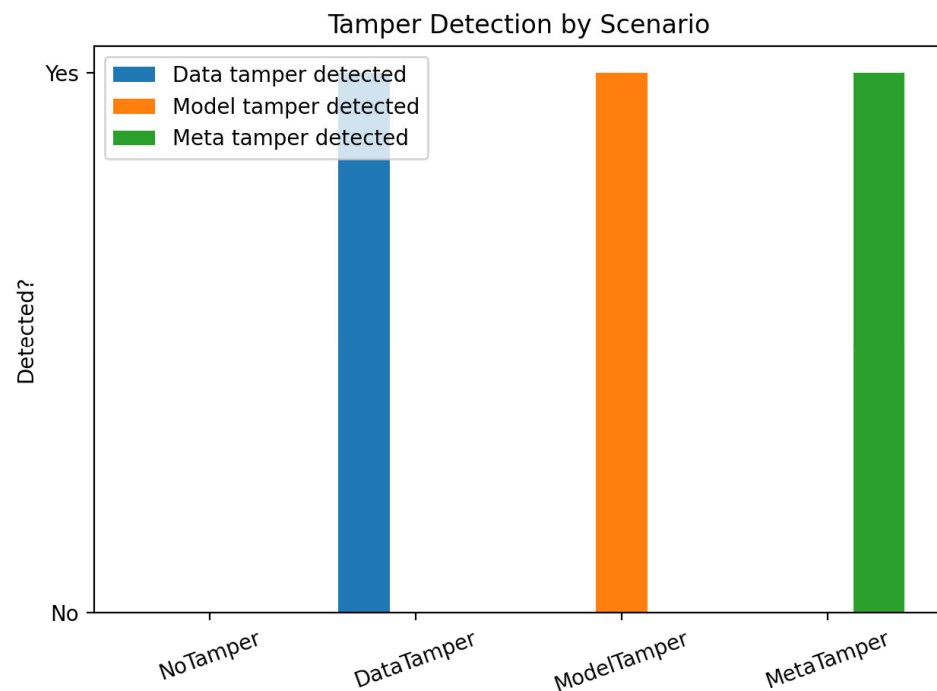


Figure 3. Tamper Detection.

Figure 3 and Table 4 above clearly show that the detection of tamper attempts is selective with no false positives.

6.2. Data Injection Scenarios

In data injection experiments, unauthorized data batches that lacked corresponding provenance records were identified during verification. Although the framework does not prevent injection at the data ingestion level, it ensures that only data with valid, verifiable provenance records are eligible for model training. This helps to ensure that the impact of the injected data is limited, as well as aid in detecting any suspicious sources after the fact.

The findings clearly indicate a significant difference between tamper prevention and tamper detection. The proposed framework does not pretend to remove adversarial access attacks but makes sure that violations of integrity and improper use of data can be traced back. These results complement those from Section 6.1 on integrity attacks by considering unauthorized data inclusion and not their modification after data registration.

6.3. Impact on Machine Learning Performance

To test the efficacy of introducing side effects through the use of provenance verification, machine learning algorithms were trained in two ways: with and without the use of provenance verification. No noticeable difference was found in terms of general metrics such as accuracy and F-score.

This result shows that the provenance framework works independently of the learning process, meaning that the learning process is not impacted even as the data integrity is maintained. In the case of attacks, the performance of the learning process on the manipulated data was suboptimal, while the data integrity checks were done prior to the learning process.

6.4. System Overhead and Scalability Considerations

In general, the computational overhead introduced by the framework seemed to be mostly dominated by hash computation and submission of blockchain transactions. Any

latencies concerning blockchain were limited to events regarding provenance registration and did not include real-time data processing or inference.

Hash computation introduced negligible overhead compared to the overall pre-processing and training times. Figure 4 presents the verification latency breakdown, showing the relative contributions of hashing and blockchain record retrieval. Verification latency varied from 22.96 ms to 31.33 ms (Table, with the dominant portion spent on blockchain record retrieval).

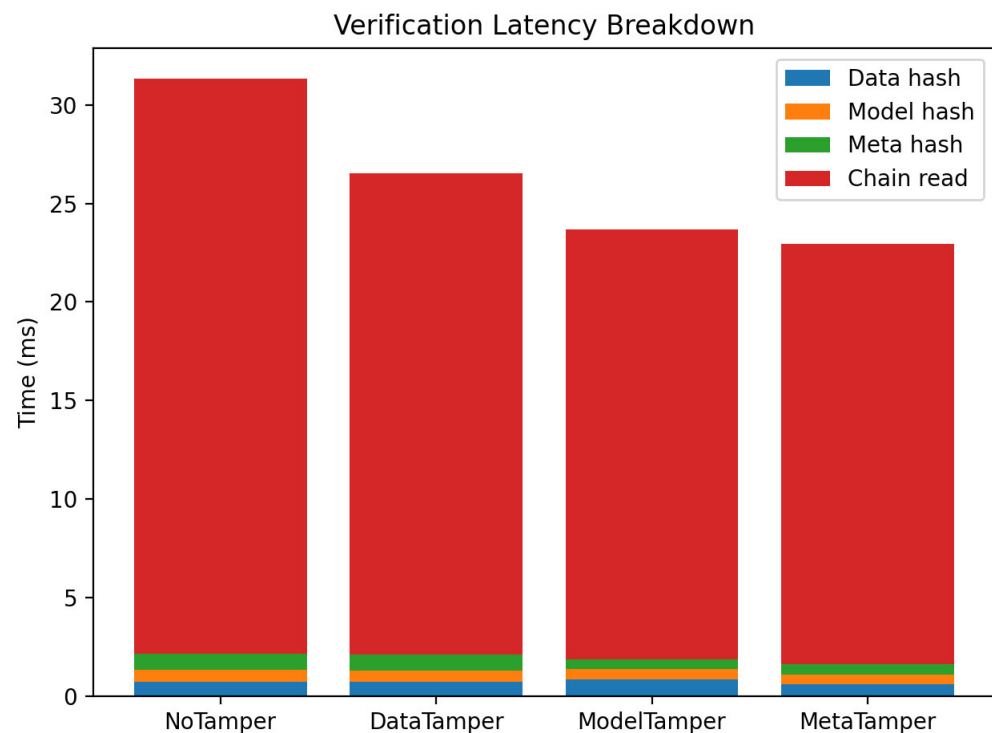


Figure 4. Latency.

The Blockchain storage overhead was low because only hashes and lightweight meta-data were stored on-chain. Off-chain storage absorbed most of the data and model sizes, confirming scalability benefits from the hybrid architecture. These results suggest that the framework can scale to large IoT datasets without imposing excessive blockchain resource consumption.

Figure 5: Gas cost of provenance record commitment, which is the same for all scenarios.

The constant gas usage across scenarios confirms that overhead in blockchain is independent of dataset size and model complexity.

6.5. Discussion

Experimentation outcomes verify that the intended design functionalities of tamper evidence, traceability, and auditability can be delivered within this framework without affecting machine learning accuracy and performance scalability. This is because it uses stronger proofs based on cryptographically connected data and models as compared to techniques that focus on partial proofs of data use and models.

However, this assessment is done in a simulation environment and does not take into account network delays, the deployment of blockchains at a network level, and constraints imposed by varying hardware. Although all these considerations can impact the absolute evaluation, the underlying integrity remains intact because it relies on cryptography.

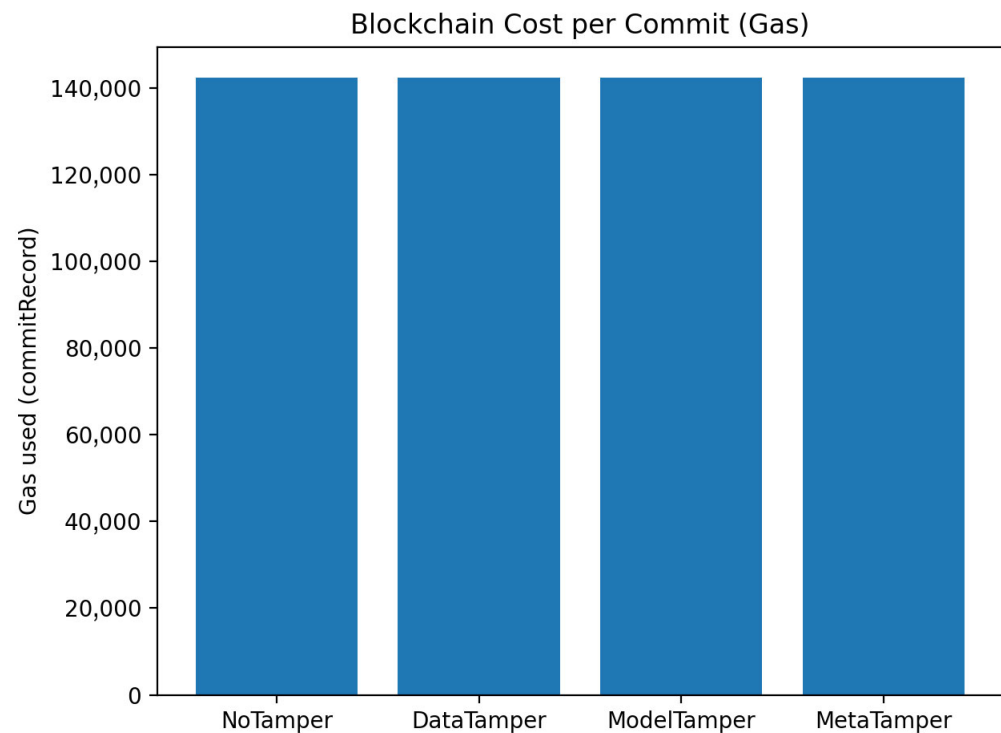


Figure 5. Gas Cost.

The quantitative findings expressed in Figures 3–5 validate the qualitative findings on security and scalability as stated above. Although the proposed framework provides process-provenance, this paper determines the detection capability of data, model, and metadata tampering as part of the experimental evaluation, and future research would constitute controlled preprocessing and feature tampering.

Provenance verification is performed before model deployment or retraining rather than during real-time inference. Therefore, detection latency primarily affects deployment workflows rather than operational decision latency. In scenarios requiring real-time decisions, periodic verification strategies may be employed to balance integrity assurance and responsiveness.

Although the current evaluation primarily examined data, model, and metadata tampering, the framework architecture also supports detection of preprocessing and feature manipulation through cryptographic linkage of process provenance records. Any unauthorized modification to preprocessing parameters or feature outputs would result in a mismatch between recomputed hashes and on-chain records. Comprehensive empirical evaluation of controlled preprocessing tampering is left for future work.

6.6. Network and Consensus Considerations

In real-world IoT deployments, network bandwidth limitations and consensus protocol characteristics may introduce additional latency during provenance registration and verification. Low-data-rate networks may increase the time required to transmit hashes and metadata to blockchain nodes. Similarly, consensus mechanisms with longer block times may delay confirmation of provenance records. These factors affect operational performance but do not compromise the integrity guarantees of the framework, which rely on cryptographic verification.

7. Security and Performance Analysis

This section evaluates the security characteristics and performance behaviors of the proposed tamper-evident provenance system on the basis of the system and threat models.

This evaluation work provides complementary information to the empirical study on the proposed system.

7.1. Security Analysis

7.1.1. Tamper-Evidence Guarantee

The primary security feature offered by the framework is that of tamper evidence. An immutable hash of each batch of data, each output of the preprocessing algorithm, as well as each model is determined and marked securely in the blockchain. It is assumed that secure hash functions protect against collisions [5], implying that it is impossible to change the off-chain artifacts without changing the hash.

Because the provenance data is recorded on the append-only blockchain ledger, the malicious manipulation of past hash values is not possible without a majority consensus of the consensus players. Therefore, any unauthorized change to data, functionality, and models can now be traced through verification. This is true irrespective of the point at which the manipulation takes place in the pipeline.

7.1.2. Provenance Integrity and Traceability

Through inter-linkages of data provenance, process provenance, and model provenance by means of cryptographic references, the framework makes the entire machine learning life cycle traceable end-to-end. The dependency chain for the provenance records is traceable through explicit references to the preceding ones.

This approach avoids the problem of provenance fragmentation, a problem identified in existing approaches, in which data and model provenances were tracked separately [9]. Attempts to replace data sources, modify preprocessing operations, or use models for a purpose not documented in the provenance will be identifiable via dangling provenance links.

7.1.3. Resistance to Provenance Forgery

Additionally, use of blockchain eliminates tampering with provenance by removing dependence on a central log or a third-party intermediary. This is due to the fact that consensus verification makes it impossible for an attacker to selectively remove and alter information. This is because on-chain storage of provenance metadata is respected even in case of compromised storage on other chains.

This is especially crucial in a multi-stakeholder IoT setting, where administrative control is distributed and there are limited assumptions about trusting entities [6].

7.1.4. Limitations and Security Scope

The framework does not target the prevention of malicious actions, including unauthorized access, denial-of-service attacks, or crafting malicious input. It rather ensures that any malicious activity has visible side effects. The framework views cryptographic primitive attacks, as well as blockchain-based consensus, as out-of-scope threats, as with most other blockchain-based security studies [8].

7.2. Performance Analysis

7.2.1. Computational Overhead

The major amount of computation, after the introduction of the framework, comes because of the computation of the cryptographic hash function along with the verification of the provenance. The computation required in hashes varies in proportion to the size of the artifact. Therefore, the new amount of computation due to the framework is negligible.

The verification tasks include hash recreation and blockchain search, and they are associated with constant and predictable costs.

7.2.2. Storage Overhead

The on-chain storage overhead is low as well, as the blockchain only keeps the hashes and metadata of data or models, and not the data and models themselves. The cost of storing one record of provenance is constant irrespective of data/model sizes.

The data and model size is entirely absorbed by the off-chain storage; this makes it scalable to a large IoT dataset. It is based on the best practice of design for a blockchain system [8].

7.2.3. Latency Considerations

The latency in the blockchain transaction affects the registration of provenance, but it has no effect on real-time inference and collection, since it proceeds asynchronously. Additionally, it has no significant effect on verification latency since it involves hash computation and access to the ledger.

Batching provenance entries, or even permissioned blockchains, can help to reduce latency even further without sacrificing security assurances in actual implementation.

7.2.4. Scalability Analysis

This is scalable with the number of data batches and updates, as opposed to the size of the data. Given the compact nature of the provenance record, the size of the blockchain will linearly scale with the number of Registered Artifacts. This is not a problem within the context of the IoT, especially where a permissioned blockchain is used.

7.2.5. Hash Computation Cost

Hash computation for large machine learning models may introduce computational overhead proportional to file size. However, hashing is performed only during artifact registration or verification and does not affect real-time inference. Since cryptographic hash functions operate in linear time with respect to input size, the cost remains predictable.

For very large models, incremental hashing or chunk-based verification can be employed to reduce memory requirements. Additionally, hashing can be executed asynchronously, allowing deployment workflows to proceed without blocking operational tasks.

7.3. Summary

It has been confirmed that the proposed framework ensures high tamper evidence, traceability, and resilience against attempts to forge the prove of origin with a modest and predictable overhead. Consequently, the proposed framework achieves a balanced performance and a high level of protection due to the complementary nature of the provided integrity and the storage architecture presented above.

8. Limitations

This study has several limitations. The evaluation was conducted in a simulated environment using synthetic datasets, which may not fully capture the complexity of real-world IoT deployments. Network latency, bandwidth constraints, and variations in blockchain consensus protocols were not explicitly modeled. Additionally, the experiments focused on integrity attacks and did not address adversarial machine learning threats or denial-of-service scenarios.

These limitations do not affect the conceptual validity of the proposed framework but indicate areas for future work, including validation with real IoT datasets and deployment in heterogeneous environments.

9. Conclusions and Future Work

This article introduced a tamper-evident provenance framework for the data and models used in IoT-based machine learning systems, with a blockchain-based off-chain approach. The need for a tamper-evident provenance system came from a lack of trust, traceability, and auditability in the current ML pipelining, prompting the introduction of this method to ensure the integrity issue posed by data tampering, preprocessing, and the modification of ML models. The system uses the blockchain-based off-chain approach for the efficient management of all machine learning processes.

The proposed methodology securely chains together the data batches from the IoT applications, the results of the preprocessing stage, and machine learning models to be able to have integrity checks and traceability analysis. The simulation test results showed that the system effectively identifies anomalies in the integrity of the data at various points in the processing chain and results in very low overhead in computation and storage. Most importantly, the provenance modules are agnostic to the machine learning phase and do not result in a deterioration in the model accuracy during normal operations.

On a general note, the contribution of this work is in line with the emerging trend of research in the area of blockchain-enabled trusted ML pipelines, and it shows how primitives of the former can be used in the latter to improve trust and accountability, and not just be used as supporting infrastructure. The work is valuable in multi-stakeholder and safety-critical applications of IoT.

There are a few areas that are still to be explored in future work. First, it is possible to extend this framework to deal with federated learning or distributed learning, where tracing provenance might be more complicated. Second, this suggests that a combination of explainable AI with these provenance chains might provide additional benefits from a proof-of-concept standpoint. Third, any form of performance optimization with regard to consensus methods that are light-weight, as well as provenance calculation from the edge level, is worth investigating in more detail. Fourth, it is worth seeing this system in a bigger picture deployment in a real-world IoT setting.

Author Contributions: Methodology, S.S.; Software, S.S. and G.A.M.; Investigation, A.K.; Writing—review & editing, S.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding .

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data used in this study were synthetically generated to simulate realistic IoT sensor behavior. The datasets were inspired by publicly available IoT datasets (e.g., TON_IoT) without using any real-world data. No proprietary or sensitive information was involved.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Gubbi, J.; Buyya, R.; Marusic, S.; Palaniswami, M. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Gener. Comput. Syst.* **2013**, *29*, 1645–1660. [[CrossRef](#)]
2. Roman, R.; Lopez, J.; Mambo, M. Mobile edge computing: A survey and analysis of security threats and challenges. *Future Gener. Comput. Syst.* **2018**, *78*, 680–698. [[CrossRef](#)]
3. Sicari, S.; Rizzardi, A.; Grieco, L.A.; Coen-Porisini, A. Security, privacy and trust in Internet of Things: The road ahead. *Comput. Netw.* **2015**, *76*, 146–164.
4. Sculley, D.; Holt, G.; Golovin, D.; Davydov, E.; Phillips, T.; Ebner, D.; Chaudhary, V.; Young, M.; Crespo, J.-F.; Dennison, D. Hidden technical debt in machine learning systems. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Montreal, QC, Canada, 7–12 December 2015.

5. Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008. Available online: <https://bitcoin.org/bitcoin.pdf> (accessed on 28 December 2025).
6. Dorri, A.; Kanhere, S.S.; Jurdak, R. Blockchain in Internet of Things: Challenges and solutions. *arXiv* **2016**, arXiv:1608.05187. [[CrossRef](#)]
7. Chen, Y.; Ding, S.; Xu, Z.; Zheng, H.; Yang, S. Blockchain-based medical records secure storage and medical service framework. *J. Med. Syst.* **2019**, *43*, 5. [[CrossRef](#)] [[PubMed](#)]
8. Xu, X.; Weber, I.; Staples, M. *Architecture for Blockchain Applications*; Springer: Cham, Switzerland, 2019.
9. Hasan, R.; Sion, R.; Winslett, M. The case of the fake Picasso: Preventing history forgery with secure provenance. In Proceedings of the USENIX Security Symposium FAST, Washington, DC, USA, 31 October 2009.
10. Kahn, R.; Wilensky, R. A framework for distributed digital object services. *Int. J. Digit. Libr.* **2006**, *6*, 115–123. [[CrossRef](#)]
11. Reyna, A.; Martín, C.; Chen, J.; Soler, E.; Díaz, M. On blockchain and its integration with IoT. *Future Gener. Comput. Syst.* **2018**, *88*, 173–190. [[CrossRef](#)]
12. Novo, O. Blockchain meets IoT: An architecture for scalable access management in IoT. *IEEE Internet Things J.* **2018**, *5*, 1184–1195. [[CrossRef](#)]
13. Christidis, K.; Devetsikiotis, M. Blockchains and smart contracts for the Internet of Things. *IEEE Access* **2016**, *4*, 2292–2303. [[CrossRef](#)]
14. Abedjan, Z.; Golab, L.; Naumann, F. (Eds.) *Data Profiling: Synthesis Lectures on Data Management*; Morgan & Claypool Publishers: San Rafael, CA, USA, 2018; Volume 12.
15. Lee, H.-C.; Lee, S.-W. Provenance-Based Trust-Aware Requirements Engineering Framework for Self-Adaptive Systems. *Sensors* **2023**, *23*, 4622. [[CrossRef](#)] [[PubMed](#)]
16. Gundersen, O.E.; Kjensmo, S. State of the Art: Reproducibility in Artificial Intelligence. *Proc. AAAI Conf. Artif. Intell.* **2018**, *32*, 1644–1651. [[CrossRef](#)]
17. Kim, H.; Park, J.; Bennis, M.; Kim, S. Blockchain on-device federated learning. *IEEE Commun. Lett.* **2019**, *24*, 1279–1283. [[CrossRef](#)]
18. Lu, Y.; Huang, X.; Dai, Y.; Maharjan, S.; Zhang, Y. Blockchain and federated learning for privacy-preserved data sharing. *IEEE Trans. Ind. Inform.* **2020**, *16*, 4177–4186.
19. Ocean Protocol Foundation. *Ocean Protocol: A Decentralized Data Exchange Protocol*; Ocean Protocol Foundation: Singapore, 2019.
20. Lin, J.; Yu, W.; Zhang, N.; Yang, X.; Zhang, H.; Zhao, W. A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications. *IEEE Internet Things J.* **2017**, *4*, 1125–1142. [[CrossRef](#)]
21. Moustafa, N.; Slay, J. TON_IoT datasets: A new generation of realistic IoT and IIoT datasets for data-driven intrusion detection systems. *IEEE Access* **2020**, *8*, 165130–165150.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.